Vorlesung: Grundlagen der Computergrafik



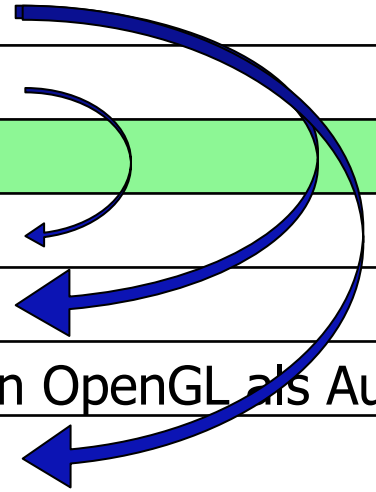
Kapitel 4:

WebGL und Three.js

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier

Geplanter Verlauf der Vorlesung

13.04.2026	Einleitung, Bild- und Videoformate
20.04.2026	Rastering, Zeichnen von Linien
27.04.2026	Transformationen + Projektionen
04.05.2026	SVG (Linien, Transformation, Animation)
11.05.2026	Beleuchtung
18.05.2026	Polygonnetze + Tesselation
25.05.2026	Pfingstmontag
01.06.2026	OpenGL (WebGL, Three.JS)
08.06.2026	Shader in WebGL/ThreeJS
15.06.2026	Texture Mapping (Texturen in OpenGL als Aufgabe der Übung)
22.06.2026	Radiosity + Raytracing
29.06.2026	Animation (keyframing) + Rest (Kinematik, Physik, Simulation, Partikel, Schwärme)
06.07.2026	Wissenschaftliche Visualisierung, Marching Cubes
13.07.2026	**Klausur**



Kapitelübersicht

- Kapitel 4.1: Grundlagen von WebGL & Three.js
- Kapitel 4.2: Shader in Three.js
- Kapitel 4.3: Texturen in Three.js

Vorlesung: Grundlagen der Computergrafik

Kapitel 4: WebGL

Kapitel 4.1:

WebGL und Three.js Grundlagen

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



OPENGL

OpenGL



- OpenGL ist eine Bibliothek (Application Program Interface API) zum Rendern einer Szene.
- Die Bibliotheksfunktionen sind so einfach gehalten, dass sich zahlreiche Operationen direkt durch die Graphik-Hardware ausführen lassen.
- Ursprünglich wurde die OpenGL Spezifikation von Silicon Graphics entwickelt, siehe www.opengl.org
- Die OpenGL Bibliothek bietet **keine** Funktionen
  - zur Kommunikation mit der graphischen Benutzeroberfläche des Betriebssystems
  - zum Drucken
  - zum Laden externer Szenen-Beschreibungen oder Geometrien
  - zum Verwalten und Erstellen einer Objekt-Hierarchie für die Szene.
  - Objekte wie Kugeln und Tori, die nicht aus Polygonen erstellt werden müssen

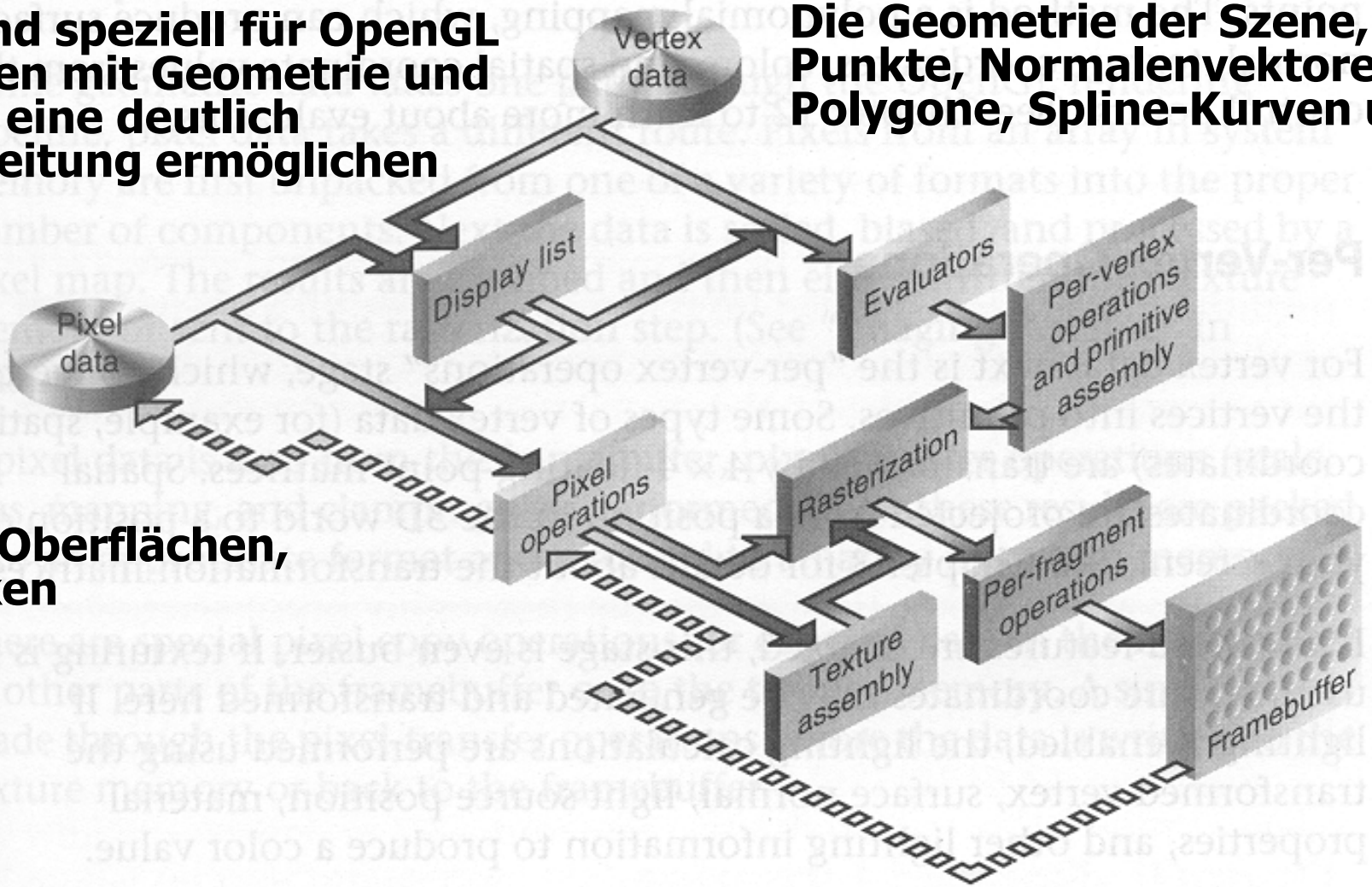
Der Rendering- Prozess von OpenGL

Display Liste

Display Listen sind speziell für OpenGL vorbereitete Listen mit Geometrie und Texturdaten, die eine deutlich schnellere Bearbeitung ermöglichen

Vertex Daten

Die Geometrie der Szene, also die Punkte, Normalenvektoren, Linien, Polygone, Spline-Kurven und Flächen



Pixel Daten

Texturen für die Oberflächen, Schablonenmasken

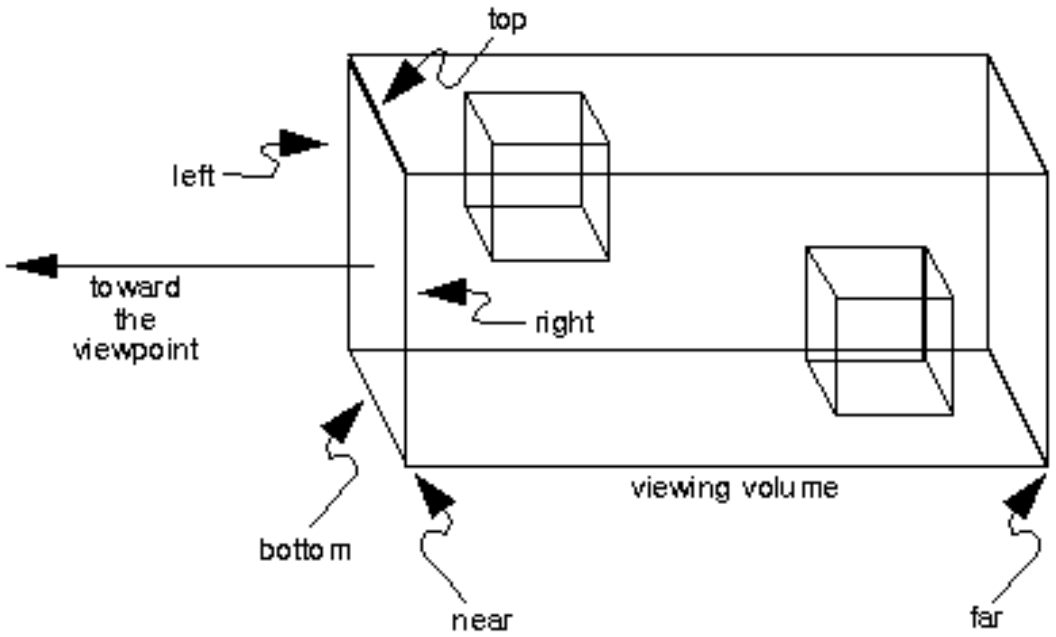
Beispiel für 2D in OpenGL

```
glClearColor (0.0, 0.0, 0.0, 0.0);
glClear (GL_COLOR_BUFFER_BIT);

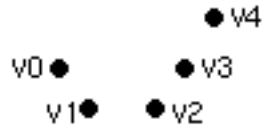
glOrtho(0.0, w, 0.0, h, -1.0, 1.0);

glColor3f (1.0, 1.0, 1.0);
glBegin(GL_POLYGON);
    glVertex3f (0.25, 0.25, 0.0);
    glVertex3f (0.75, 0.25, 0.0);
    glVertex3f (0.75, 0.75, 0.0);
    glVertex3f (0.25, 0.75, 0.0);
glEnd();
glFlush();
```

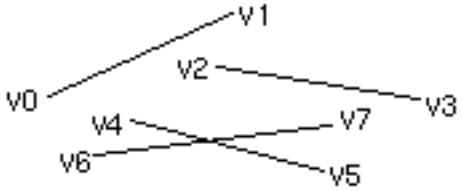
Parallele Projektion



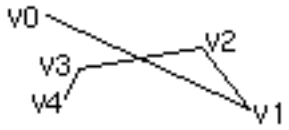
OpenGL Datenstrukturen



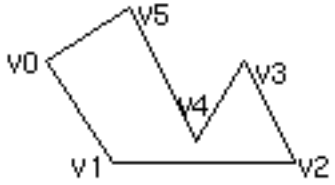
GL_POINTS



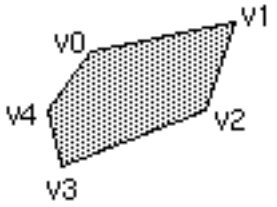
GL_LINES



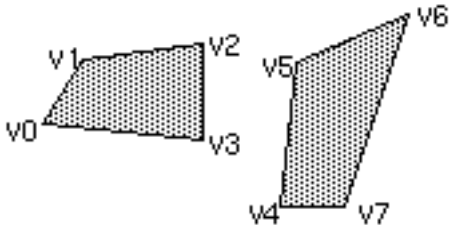
GL_LINE_STRIP



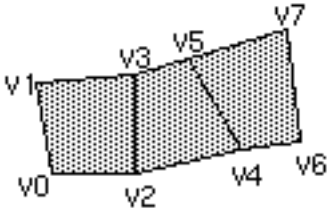
GL_LINE_LOOP



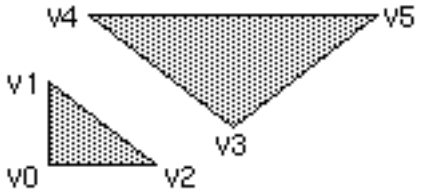
GL_POLYGON



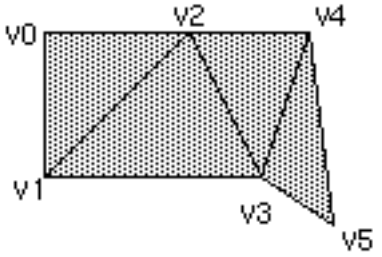
GL_QUADS



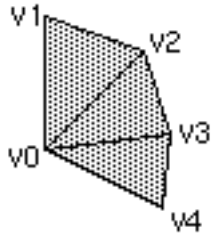
GL_QUAD_STRIP



GL_TRIANGLES



GL_TRIANGLE_STRIP

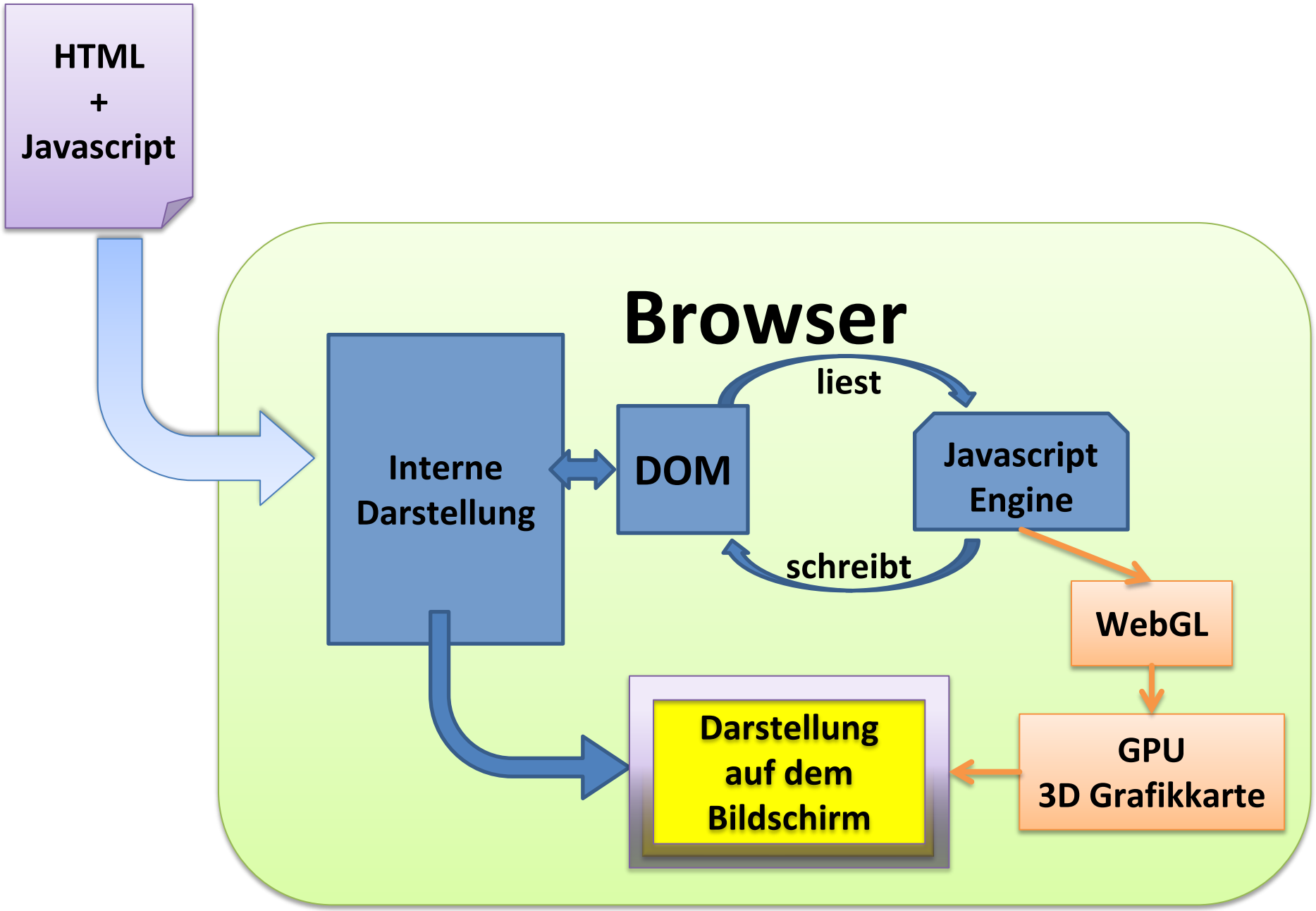


GL_TRIANGLE_FAN

WEBGL



- WebGL ≈ OpenGL ES 2.0 (embedded systems)
- Wird von allen gängigen Browsern unterstützt
- API auf niedriger Abstraktionsebene
- Die meisten Entwickler verwenden daher auf WebGL aufbauende APIs
- Beispiel: https://webglsamples.org/


HTML
+
Javascript
Browser
Interne
Darstellung
DOM
liest
schreibt
Javascript
Engine
WebGL
GPU
3D Grafikkarte
Darstellung
auf dem
Bildschirm

THREE.JS

Three.js

- Verwendet bekannte 3D-Konzepte auf höherer Abstraktionsebene als WebGL
- Viele zusätzliche Funktionalitäten: Mathematik, Graphische Objekte, Animation, Interaktion, Laden von Dateien
- Objekt-orientiert
- Beispiele: http://threejs.org
- Quellkode: https://github.com/mrdoob/three.js/

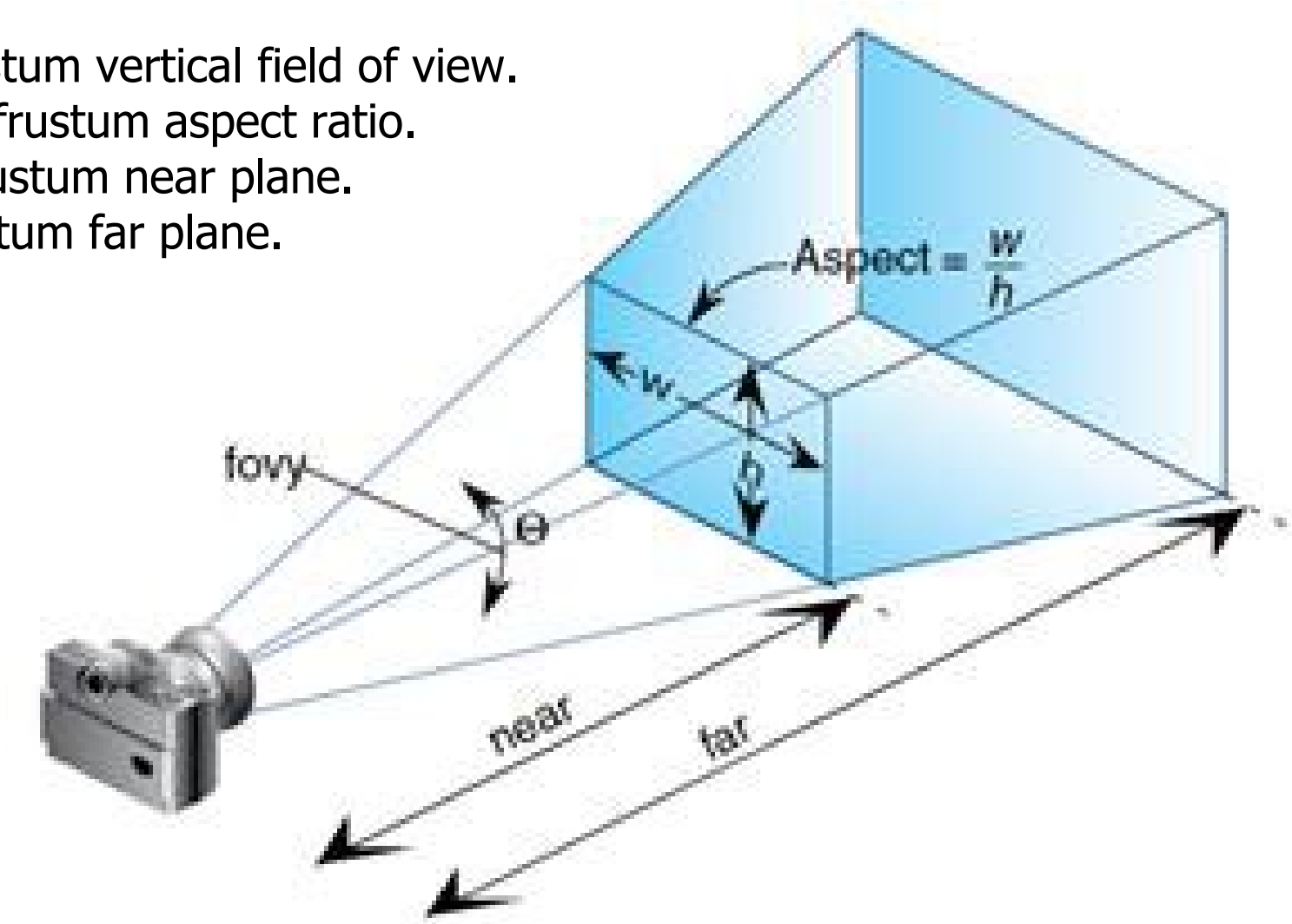
```
<html>
<head> <title>A Simple Three.js Page</title>
<script src="../libs/Three.js"></script>
<script>
function onLoad()
{    // Grab our container div
    var container = document.getElementById("container");
  // Create the Three.js renderer, add it to our div
   var renderer = new THREE.WebGLRenderer();
    renderer.setSize(container.offsetWidth, container.offsetHeight);
    container.appendChild( renderer.domElement );
  // Create a new Three.js scene
   var scene = new THREE.Scene();
 // Create a camera and add it to the scene
    var camera = new THREE.PerspectiveCamera( 45,
                        container.offsetWidth / container.offsetHeight, 1, 4000 );
      camera.position.set( 0, 0, 3.3333 );
      scene.add( camera );
  // Now, create a rectangle and add it to the scene
      var geometry = new THREE.PlaneGeometry(1, 1);
      var mesh = new THREE.Mesh( geometry, new THREE.MeshBasicMaterial( ) );
      scene.add( mesh );
    // Render it
    renderer.render( scene, camera );
}
</script>
</head>
<body onLoad="onLoad();">
    <div id="container"
            style="width:500px; height:500px; background-color:#000000"></div>
</body>
</html>
```

Example2-1

```
function onLoad()
{   // Grab our container div
    var container = document.getElementById("container");
    // Create the Three.js renderer, add it to our div
    var renderer = new THREE.WebGLRenderer();
    renderer.setSize(container.offsetWidth, container.offsetHeight);
    container.appendChild( renderer.domElement );
    // Create a new Three.js scene
    var scene = new THREE.Scene();
    // Create a camera and add it to the scene
    var camera = new THREE.PerspectiveCamera( 45,
                                container.offsetWidth / container.offsetHeight, 1, 4000 );
    camera.position.set( 0, 0, 3.3333 );
    scene.add( camera );
    // Now, create a rectangle and add it to the scene
    var geometry = new THREE.PlaneGeometry(1, 1);
    var mesh = new THREE.Mesh( geometry, new THREE.MeshBasicMaterial( ) );
    scene.add( mesh );
    // Render it
    renderer.render( scene, camera );
}
```

PerspectiveCamera(fov, aspect, near, far)

- fov — Camera frustum vertical field of view.
  aspect — Camera frustum aspect ratio.
  near — Camera frustum near plane.
  far — Camera frustum far plane.



Example2-2

```
// Create a directional light to show off the object
var light = new THREE.DirectionalLight( 0xffffff, 1.5);
light.position.set(0, 0, 1);
scene.add( light );

// Create a shaded, texture-mapped cube and add it to the scene

   // First, create the texture map
   var mapUrl = "../images/molumen_small_funny_angry_monster.jpg";
   var map = THREE.ImageUtils.loadTexture(mapUrl);   // deprecated !!!!
   // Now, create a Phong material to show shading; pass in the map
   var material = new THREE.MeshPhongMaterial({ map: map });
   // Create the cube geometry
   var geometry = new THREE.CubeGeometry(1, 1, 1);
   // And put the geometry and material together into a mesh
   cube = new THREE.Mesh(geometry, material);
    // Turn it toward the scene, or we won't see the cube shape!
    cube.rotation.x = Math.PI / 5;
    cube.rotation.y = Math.PI / 5;
    // Add the cube to our scene
    scene.add( cube );
    // Add a mouse up handler to toggle the animation

addMouseHandler();
// Run our render loop
run();
```

Animation in Example2-2

```
function run()
 {     // Render the scene
       renderer.render( scene, camera );
       // Spin the cube for next frame
       if (animating)
          { cube.rotation.y -= 0.01;
          }
       // Ask for another frame
       requestAnimationFrame(run);
 }
```

Interaktion in Example2-2

```
function addMouseHandler()
 {  var dom = renderer.domElement;
    dom.addEventListener( 'mouseup',
                                    onMouseUp, false);
 }

function onMouseUp(event)
  {  event.preventDefault();
     animating = !animating;
  }
```

Eigenes Polygonnetz erzeugen

```
// First, create MeshFaceMaterial for coloring each face
    var mats = [];
    mats.push(new THREE.MeshBasicMaterial({ color: 0x009e60  }));
    mats.push(new THREE.MeshBasicMaterial({ color: 0x0051ba  }));
    mats.push(new THREE.MeshBasicMaterial({ color: 0xffd500  }));
    mats.push(new THREE.MeshBasicMaterial({ color: 0xff5800  }));
    mats.push(new THREE.MeshBasicMaterial({ color: 0xC41E3A }));
    mats.push(new THREE.MeshBasicMaterial({ color: 0xffffff  }));

 // Create the cube geometry by hand
     var geometry = new THREE.Geometry();

    geometry.vertices.push(new THREE.Vector3( 1.0,  1.0, -1.0));
    geometry.vertices.push(new THREE.Vector3(-1.0,  1.0, -1.0));
    geometry.vertices.push(new THREE.Vector3(-1.0, -1.0, -1.0));
    // ...

    // Note: Face3( a, b, c, normal, color, materialIndex ) ACHTUNG DAS FOLGENDE DREIECK HAT FALSCHE ECKEN !!!
    geometry.faces.push(new THREE.Face3(0,1,2,null,null,0));
    // ...

 // And put the geometry and material together into a mesh
    cube = new THREE.Mesh(geometry,mats);
```

Constructor THREE.Face3()

- **Face3**(a : Integer, b : Integer, c : Integer,
  normal : Vector3, color : Color, materialIndex : Integer)
- a — Vertex A index.
  b — Vertex B index.
  c — Vertex C index.

  normal — (optional) Face normal (Vector3) or array of vertex normals. If a single vector is passed in, this sets .normal, otherwise if an array of three vectors is passed in this sets .vertexNormals

  color — (optional) Face color or array of vertex colors. If a single vector is passed in, this sets .color, otherwise if an array of three vectors is passed in this sets .vertexColors

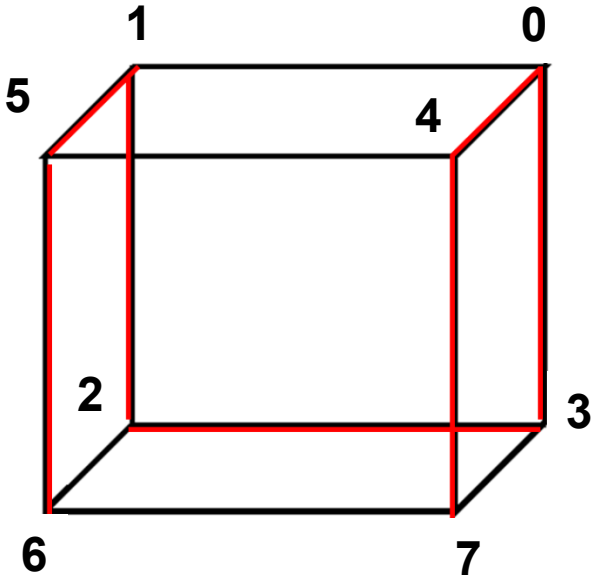
  materialIndex — (optional) which index of an array of materials to associate with the face.

Aufgabe 1

https://threejs.org/docs/

- Würfelgeometrie von Hand erzeugen

```
var geometry = new THREE.Geometry();
geometry.vertices.push(new THREE.Vector3( 1.0,  1.0, -1.0));
...
geometry.faces.push(new THREE.Face3(0,4,3,null,null,0));
...
```



https://t.ly/SumJx

https://www.st.uni-trier.de/lectures/S26/CG/webgl/aufgabe1/exercise1ace.html

Aufgabe 2

- Zauberwürfel aus kleinen Würfeln zusammenbauen

```
var newCube = cube.clone(); // cube is a mesh
newCube.position.set(100,100,100);
group=new THREE.Object3D();
group.add(newCube);
```



Schattierungen mit Lambert-Beleuchtungsmodell

```
var mats = [];
mats.push(new THREE.MeshLambertMaterial({ color: 0x009e60  }));
mats.push(new THREE.MeshLambertMaterial({ color: 0x0051ba  }));
…

// mit Eckennormalen ergeben sich Schattierungen
// Nehme Ortsvektor als Eckennormale

var normalenA = [geometry.vertices[0].clone().normalize(),
                 geometry.vertices[1].clone().normalize(),
                 geometry.vertices[2].clone().normalize()];

geometry.faces.push(new THREE.Face3(0,1,2,normalenA,null,0));
```